CHAPTER 1

DESIGN INSTAGRAM

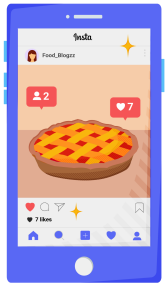
1 System Design: Instagram 2
2 Requirements of Instagram's Design 3
3 Design of Instagram 7
4 Detailed Design of Instagram 10
5 Quiz on Instagram's Design 15

Explore Instagram's System Design, covering API design, storage schema, and timeline generation using pull, push, and hybrid approaches.

1 System Design: Instagram

1.1 What is Instagram?

Instagram is a free social networking application that allows users to upload photos and short videos. Users can add a caption for each post and utilize hashtags or location-based geotags to index them and make them searchable within the application. Posts appear in a user's followers' newsfeeds, and content tagged with hashtags or geotags can be accessed by the broader public, helping users reach a wider audience. Users can also choose to make their profile private, which limits access to those who have chosen to follow them.



> **Note:** As Instagram's user base grows globally, the demand for resources such as servers, databases, and content delivery networks (CDNs) also increases. Instagram must continously optimize its backend architecture to meet the increasing demands of millions of daily active users. Efficiently managing user growth, predicting future resouce requirements, and ensuring scalability are essential for handling the massive volumes of real-time, user generated content.

The expansion in the number of users requires more resources (servers, databases, and so on) over time. Knowing the users' growth rate helps us predict the resources to scale our system accordingly. The following illustration shows the Instagram user base in different countries as of January 2022 (source: Statista).

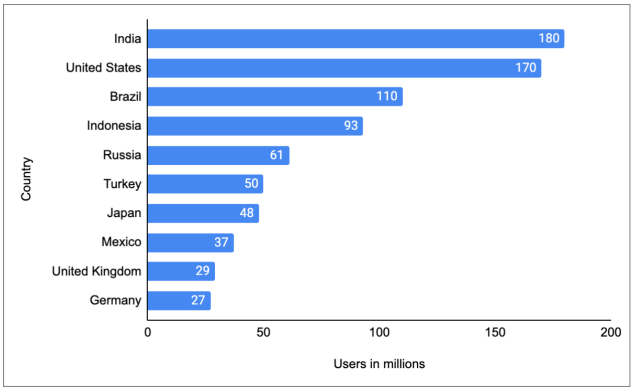


Figure 1.1: Countries with the most Instagram users

Did you know?

Instagram's seamless user experience—instant notifications, personalized feeds, and high availability is driven by a powerful system design using Redis, Cassandra, and advanced file storage. Explore how these technologies power the social media giant and inspire top engineering interview questions!

1.2 How will we design Instagram?

This section is designed for aspiring software engineers or those preparing for system design interviews. We have divided the design of Instagram into four lessons:

1. **Requirements**: This lesson will put forth the functional and nonfunctional requirements of Instagram. It will also estimate the resources required to achieve these requirements.
2. **Design**: This lesson will explain the workflow and usage of each component, its high-level design, API design and database schema.
3. **Detailed design**: In this lesson, we'll explore the components of our Instagram design in detail, discuss various approaches to generating timelines, and evaluate our proposed design.
4. **Quiz**: This lesson will test our understanding of the Instagram design.

- How can Instagram handle feed generation and messaging to deliver personalized user feeds, integrate notifications, and ensure reliable storage and retrieval of feed data for optimal photo-sharing functionality?
- How do load balancers, rate limiters, and Kafka help ensure Instgram's performance during peak activity, streamlining messaging and handling millions of user requests with low latency and optimal user experience?
- Why is selecting the right data store (NoSQL vs. relational databases) crucial for managing user data, feed data, and high volumes of requests while maintaining scalability and reliability?
- What considerations must Instagram address when implementing file and object storage (e.g., Amazon S3), and how do sharding and replication impact large-scale photo-sharing?

Let's start by understanding the requirements for designing our Instagram system, using practical insights on distributed systems and API optimization. In the next lesson, we will provide resource estimations.

2 Requirements of Instagram's Design

2.1 Requirements

We'll concentrate on some important features of Instagram to make this design simple. Let's list down the requirements for our system:

Functional requirements

- **Post photos and videos**: The users can post photos and videos on Instagram.
- **Follow and unfollow users**: The users can follow and unfollow other users on Instagram.
- **Like or dislike posts**: The users can like or dislike posts of the accounts they follow.
- **Search photos and videos**: The users can search photos and videos based on captions and location.
- **Generate news feed**: The users can view the news feed consisting of the photos and videos (in chronological order) from all the users they follow. Users can also view suggested and promoted photos in their news feed.

Non-functional requirements

- **Scalability**: The system should be scalable to handle millions of users in terms of computational resources and storage.
- **Latency**: The latency to generate a news feed should be low.
- **Availability**: The system should be highly available.

- **Durability:** Any uploaded content (photos and videos) should never get lost.
- **Consistency**: We can compromise a little on consistency. It is acceptable if the content (photos or videos) takes time to show in followers' feeds located in a distant region.
- **Reliability**: The system must be able to tolerate hardware and software failures.

Having understood the requirements of Instagram's design, answer the following question:
Suppose we wanted to implement a time-based expiration feature for the content on Instagram (like Snapchat). What functional requirements would we need to add or remove to reflect this feature?

2.2 Resource estimation

Our system is read-heavy because service users spend substantially more time browsing the feeds of others than creating and posting new content. Our focus will be to design a system that can fetch the photos and videos on time. There is no restriction on the number of photos or videos that users can upload, meaning efficient storage management should be a primary consideration in the design of this system. Instagram supports around a billion users globally who share 95 million photos and videos on Instagram per day. We 'll calculate the resources and design our system based on the requirements.
Let's assume the following:

- We have 1 billion users, with 500 million as daily active users.
- Assume 60 million photos and 35 million videos are shared on Instagram per day.
- We can consider 3 MB as the maximum size of each photo and 150 MB as the maximum size of each video uploaded on Instagram.
- On average, each user sends 20 requests (of any type) per day to our service.

Storage estimation

We need to estimate the storage capacity, bandwidth, and the number of servers to support such an enormous number of users and content.
The storage per day will be:
60 million photos/day $\times$ 3 MB = 180 TB/day
35 million videos/day $\times$ 150 MB = 5250 TB/day
Total content size = 180 + 5250 = 5430 TB
The total space required for a year:
5430 TB/day $\times$ 365 (days a year) = 1981950 TB = 1981.95 PB

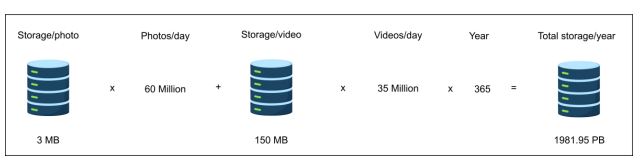


Figure 1.2: Total storage required by the Instagram storage system for a year

Besides photos and videos, we have ignored comments, status sharing data, and so on. Moreover , we also have to store users' information and post metadata, for example, userID, photo, and so on. So , to be precise, we need more than 5430 TB/day, but to keep our design simple, let's stick to the 5430 TB/day.

Bandwidth estimation

According to the estimate of our storage capacity, our service will get 5430 TB of data each day, which will give us:
$\frac{5430 \text{ TB}}{24 \times 60 \times 60} = \frac{5430 \text{ TB}}{86400 \text{ sec}} \approx 62.84 \text{ GB/s} \approx 502.8 \text{ Gbps}$

Since each incoming photo and video needs to reach the users' followers, let's say the ratio of readers to writers is 100 : 1. As a result, we need 100 times more outgoing bandwidth than incoming bandwidth. Let's assume the following:
Incoming bandwidth ~= 502.8 Gbps
Required outgoing bandwidth ~= 100×502.8 Gbps ≈ 50.28 Tbps

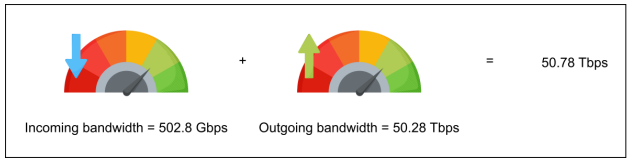


Figure 1.3: Total bandwidth required

Outgoing bandwidth is fairly high. We can use compression to reduce the media size substantially. Moreover , we'll place content close to the users via CDN and other caches in IXP and ISPs to serve the content at high speed and low latency.

Server count

We will estimate server count in two ways: assuming a uniform request distribution and for the peak load. In the first method, we will assume that all the requests are distributed uniformly over a day. In some sense, that might be considered the best case for our service because the load will be ideally distributed. In the second method, we assume that all the requests of a day show up simultaneously. Such a calculation gives us an upper bound on the number of servers.
One can argue that both ways might be far from the reality. However, these calculations give us lower and upper bounds on the server estimates. We can tighten these bounds by changing our assumptions. For example, if we know (or our interviewer tells us) that no more than 20% of the daily requests can show up simultaneously, then peak server estimates will change accordingly.

Server count assuming uniform request distribution We need to handle concurrent requests from 500 million daily active users, which are uniformly distributed over 24 hours. Let 's assume that a typical Instagram server handles 64,000 requests per second.

- Requests by each user per day = 20
- Total requests per day = 500 million $\times$ 20
- Requests per second = $\frac{500 \text{ million} \times 20}{24 \times 60 \times 60} \approx 116$ K

The required servers are estimated as follows:

$$\text{Servers needed for uniform requests} = \frac{\text{Number of requests/second}}{\text{RPS of server}}$$

$$\text{Servers needed for uniform requests} = \frac{116 \text{ K}}{64{,}000} \approx 2 \text{ servers}$$

This calculation shows that we need only 2 servers to handle queries in our Instagram system if the request load is uniformly distributed.

Server count assuming peak load Considering our assumption of using daily active users as a proxy for the number of requests per second to find the number of servers for peak load times, we get 500 million requests per second. Then , we use the following formula to calculate the number of servers:

$$\text{Servers needed at peak load} = \frac{\text{Number of requests/second}}{\text{RPS of server}}$$

Figure 1.4: Number of servers required for the Instagram system under unifrom request distribution assumption

$$\text{Servers needed at peak load} = \frac{500 \text{ million}}{64{,}000} = 7812.5 \approx 8\text{K servers}$$

Figure 1.5: Number of servers required for the Instagram system under peak load assumption

Note: Concurrent requests significantly impact the number of required servers compared to requests spread over time. For a more in-depth exploration of refining peak load assumptions, refer to the "Back-of-the-envelope Calculations" chapter.

2.3 Try it yourself

Let's analyze how the number of photos per day affects the storage and bandwidth requirements. For this purpose, try to change values in the following table to compute the estimates. We have set 20 requests by each user per day in the following calculation:

2.4 Building blocks we will use

In the next lesson, we'll focus on the high-level design of Instagram. The design will utilize many building blocks that have been discussed in the initial chapters also. We 'll use the following building blocks in our design:

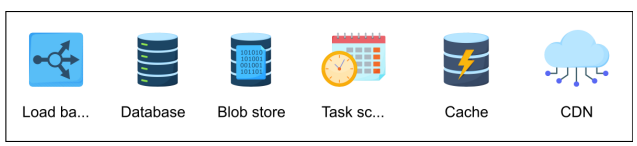


Figure 1.6: Building blocks used in Instagram design

- A **load balancer** at various layers will ensure smooth request distribution among available servers.
- A **database** is used to store the user and accounts metadata and relationship among them.
- **Blob storage** is needed to store the various types of content such as photos, videos, and so on.
- A **task scheduler** schedules the events on the database such as removing the entries whose time to live exceeds the limit.
- A **cache** stores the most frequent content-related requests.
- **CDN** is used to effectively deliver content to end users which reduces delay and burden on end-servers.

In the next lesson, we'll discuss the high-level design of the Instagram system.

3 Design of Instagram

3.1 High-level design

Our system should allow us to upload, view, and search images and videos at a high level. To upload images and videos, we need to store them, and upon fetching, we need to retrieve the data from the storage. Moreover , the users should also be allowed to follow each other.

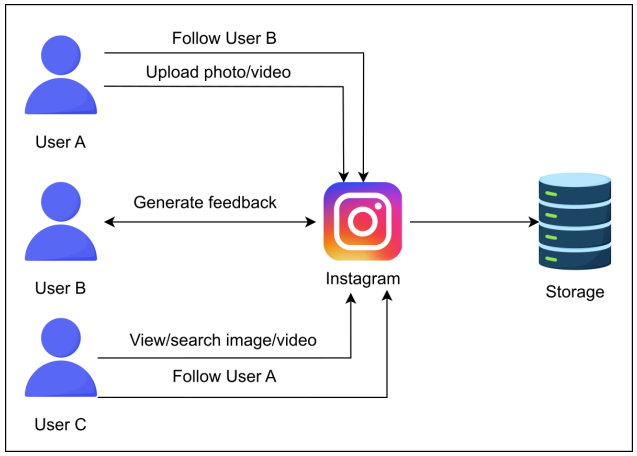


Figure 1.7: The high-level design of Instagram

3.2 API design

This section describes APIs invoked by the users to perform different tasks (upload, like, and view photos/videos) on Instagram. We 'll implement REST APIs for these tasks. Let's develop APIs for each of the following features:

- Post photos and videos
- Follow and unfollow users
- Like or dislike posts
- Search photos and videos
- Generate a news feed

All of the following calls will have a `userID`, that uniquely specifies the user performing the action. We 'll only discuss new parameters in the calls.

Post photos or videos

The POST method is used to post photos/videos to the server from the user through the `/postMedia` API. The `/postMedia` API is as follows:

```
postMedia(userID, media_type, media_file, list_of_hashtags, caption)
```

Parameter	Description
`media_type`	It indicates the type of media (photo or video) in a post.
`media_file`	It holds media file (photo or video) of a post.
`list_of_hashtags`	It represents all hashtags (maximum limit 30 hashtags) in a post.
`caption`	It is a text (maximum limit is 2,200 characters) in a user's post.

Follow and unfollow users

The `/followUser` API is used when a user follows other users on Instagram. The `/followUser` API is as follows:

```
followUser(userID, target_userID)
```

Parameter	Description
`target_userID`	It indicates the user to be followed.

The `/unfollowUser` API uses the same parameters when a user unfollows someone on Instagram.

Like or dislike posts

The `/likePost` API is used when users like someone's post on Instagram.

```
likePost(userID, post_id)
```

Parameter	Description
`post_id`	It specifies the post's unique ID.

The `/dislikePost` API uses the same parameters when a user dislikes someone's post on Instagram.

Search photos or videos

The GET method is used when the user searches any photos or videos using a keyword or hashtag. The `/searchPhotos` API is as follows:

```
searchPhotos(userID, keyword)
```

> **Note:** Instagram shows the posts with the highest reach (those with more likes and views) upon searching for a particular key. For example, if a user does a location-based search using "London, United Kingdom," Instagram will show the posts in order with maximum to minimum reach. Instead of showing all the posts, the data will be loaded upon scrolling.

Generate news feed

The GET method is used when users view their news feed through the `/viewNewsfeed` API. The `/viewNewsfeed` API is as follows:

```
viewNewsfeed(userID, generate_timeline)
```

Instagram introduced a new feature where users can filter the newsfeed generation request. Usually , the newsfeed is generated based on users' followers. But now there would be another option of favorite shows latest from the accounts chosen by the user.-->

Parameter	Description
`keyword`	It indicates the string (username, hashtag, and places) typed by the user in the search bar.

Parameter	Description
`generate_timeline`	It indicates the time when a user requests news feed generation. Instagram shows the posts that are not seen by the user between the last news feed request and the current news feed request.

3.3 Storage schema

Let's define our data model now:

Relational or non-relational database

It is essential to choose the right kind of database for our Instagram system, but which is the right choice—SQL or NoSQL? Our data is inherently relational, and we need an order for the data (posts should appear in chronological order) and no data loss even in case of failures (data durability). Moreover , in our case, we would benefit from relational queries like fetching the followers or images based on a user ID. Hence , SQL-based databases fulfill these requirements.
So, we'll opt for a relational database and store our relevant data in that database.

Define tables

On a basic level, we need the following tables:

- **Users**: This stores all user-related data such as ID, name, email, bio(An Instagram bio is a brief description of a user that appears beneath the username.), location, date of account creation, time of the last login, and so on.
- **Followers**: This stores the relations of users. In Instagram, we have a unidirectional relationship, for example, if user A accepts a follow request from user B, user B can view user A's post, but vice versa is not valid.
- **Photos**: This stores all photo-related information such as ID, location, caption, time of creation, and so on. We also need to keep the user ID to determine which photo belongs to which user. The user ID is a foreign key from the users table.
- **Videos**: This stores all video-related information such as ID, location, caption, time of creation, and so on. We also need to keep the user ID to determine which video belongs to which user. The user ID is a foreign key from the users table.

  Quiz: Question 1: Where should we store the photos and videos? **Answer:** We'll store the photos and videos in a blob store (like S3) and save the path of the photo or video in the table as it is efficient to save larger data in a distributed storage.

The following illustration visualizes the data model:

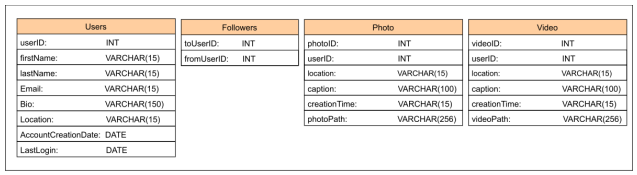


Figure 1.8: The Data model of Instagram

Data estimation

Let's figure out how much data each table will store. The column per row size (in bytes) in the following calculator shows the data per row of each table. It also calculates the storage needed for the specified counts. For example, the storage required for 500 million users is 111000 MB, 2000 MB for 250 followers of a single user, 23640 MB for 60 million photos, and 13790 MB for 35 million videos.
You can change the values in the calculator to observe the change in storage needed. This gives us an estimate of how fast data will increase in our tables.

> **Note:** Most modern services use both SQL and NoSQL stores. Instagram officially uses a combination of SQL (PostgreSQL) and No-SQL (Cassandra) databases. The loosely structured data like timeline generation is usually stored in No-SQL, while relational data is saved in SQL-based storage.

In the next lesson, we'll identify more components to tweak our design.

4 Detailed Design of Instagram

4.1 Add more components

Let's add a few more components to our design:

- **Load balancer**: To balance the load of the requests from the end users.
- **Application servers**: To host our service to the end users.
- **Relational database**: To store our data.
- **Blob storage**: To store the photos and videos uploaded by the users.

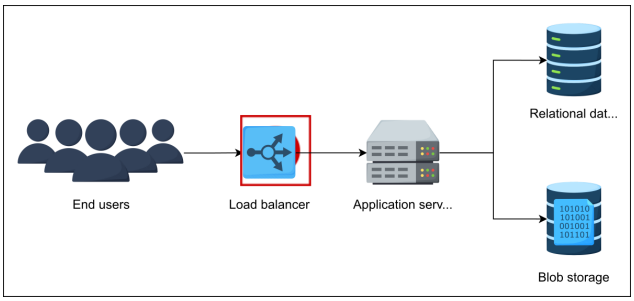


Figure 1.9: Adding components to design

4.2 Upload, view, and search a photo

The client requests to upload the photo, load balancer passes the request to any of the application servers, which adds an entry to the database. An update that the photo is stored successfully is sent to the user. If an error is encountered, the user is communicated about it as well.
The photo viewing process is also similar to the above-mentioned flow. The client requests to view a photo, and an appropriate photo that matches the request is fetched from the database and shown to the user. The client can also provide a keyword to search for a specific image.
The read requests are more than write requests and it takes time to upload the content in the system. It is efficient if we separate the write (uploads) and read services. The multiple services operated by many servers

handle the relevant requests. The read service performs the tasks of fetching the required content for the user, while the write service helps upload content to the system.
We also need to cache the data to handle millions of reads. It improves the user experience by making the fetching process fast. We 'll also opt for lazy loading(Lazy loading contributes to efficiency in the program's operation. It is ideal in use cases where network content is accessed and initialization times are to be kept at a minimum, such as in the case of web pages. For example, deferring the loading of images on a web page until they are needed can make the initial display of the web page faster. Source : Wikipedia), which minimizes the client's waiting time. It allows us to load the content when the user scrolls and therefore save the bandwidth and focus on loading the content the user is currently viewing. It improves the latency to view or search a particular photo or video on Instagram.
The updated design is as follows:

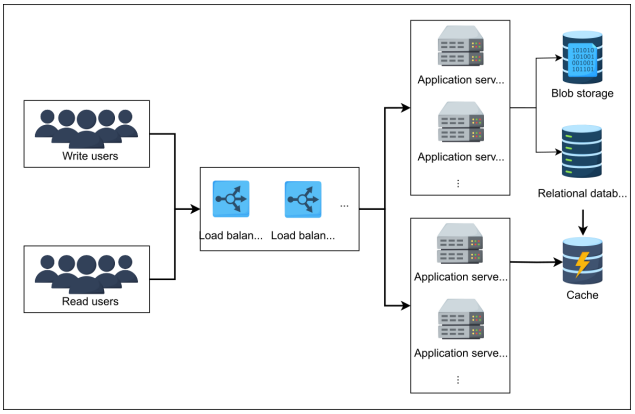


Figure 1.10: Various operations on photos

4.3 Generate a timeline

Now our task is to generate a user-specific timeline. Let 's explore various approaches and the advantages and disadvantages to opt for the appropriate strategy.

The pull approach

When a user opens their Instagram, we send a request for timeline generation. First , we fetch the list of people the user follows, get the photos they recently posted, store them in queues, and display them to the user. But this approach is slow to respond as we generate a timeline every time the user opens Instagram.

We can substantially reduce user-perceived latency by generating the timeline offline. For example, we define a service that fetches the relevant data for the user before, and as the person opens Instagram, it displays the timeline. This decreases the latency rate to show the timeline. Let 's take a look at the slides below to understand the problem and its solution.

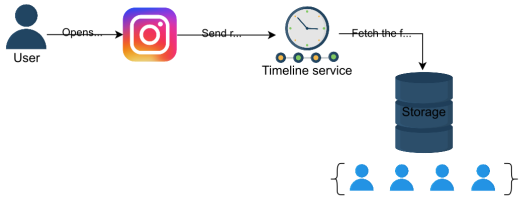
> **Quiz: Question 1:** What are the shortcomings of the pull approach? **Answer:** Instagram is a read-heavy system. Many people don't post any photos and instead just view others' posts. So, the calls we make to fetch the recent posts from every follower will usually return nothing. Hence, we should keep in mind that it is not a write-heavy system and should develop a possible solution to cater to it.

The push approach

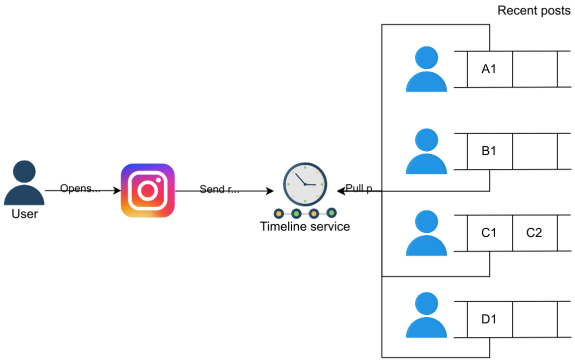
In a **push approach**, every user is responsible for pushing the content they posted to the people's timelines who are following them. In the previous approach, we pulled the post from each follower, but in the current approach, we push the post to each follower.

(a) A user opens Instagram

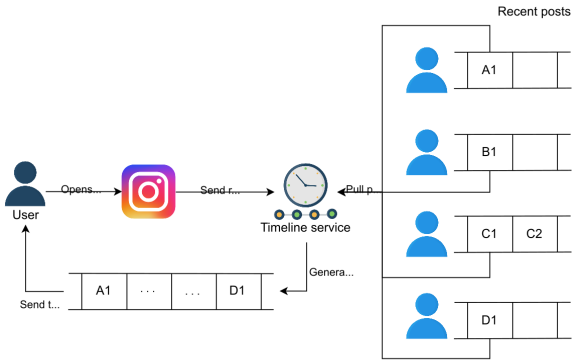
(b) A request to generate a timeline is sent to the timeline service



(a) The timeline service fetches the list of followers for that user from the storage



(b) The timeline service fetches the recent posts of the followers



(a) Display the fetched photos on the user's timeline

Now we only need to fetch the data that is pushed towards that particular user to generate the timeline. The push approach has stopped a lot of requests that return empty results when followed users have no post in a specified time.

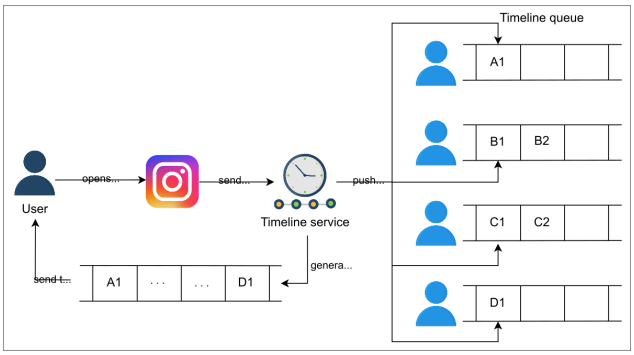


Figure 1.14: Push-based approach

> **Quiz: Question 1:** What are the shortcomings of the push approach? **Answer:** Consider an account that belongs to a celebrity, like Cristiano Ronaldo,
>
> who has over 400 million followers. So if he posts a photo or a video, we will push the links of the photo/video to 400 million+ users, which is inefficient.

Hybrid approach

Let's split our users into two categories:

- **Push-based users**: The users who have a followers count of hundreds or thousands.
- **Pull-based users**: The users who are celebrities and have followers count of a hundred thousand or millions.

The timeline service pulls the data from pull-based followers and adds it to the user's timeline. The push-based users push their posts to the timeline service of their followers so the timeline service can add to the user's timeline.

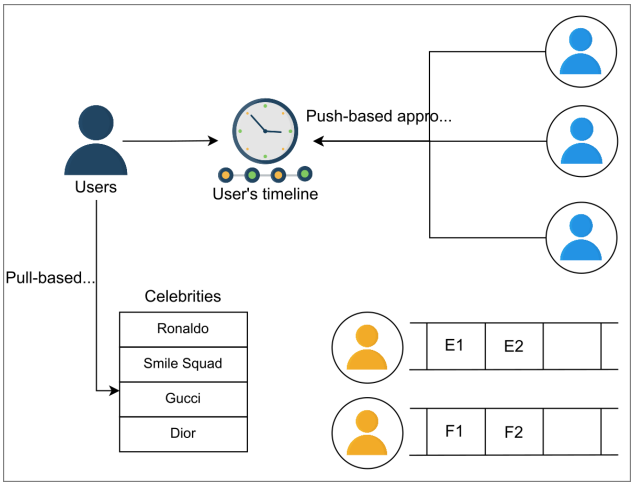


Figure 1.15: Hybrid approach

We have used the method which generates the timeline, but where do we store the timeline? We store a user's timeline against a `userID` in a key-value store. Upon request, we fetch the data from the key-value

store and show it to the user. The key is `userID`, while the value is timeline content (links to photos and videos). Because the storage size of the value is often limited to a few MegaBytes, we can store the timeline data in a blob and put the link to the blob in the value of the key as we approach the size limit.
We can add a new feature called story to our Instagram. In the story feature, the users can add a photo that stays available for others to view for 24 hours only. We can do this by maintaining an option in the table where we can store a story's duration. We can set it to 24 hours, and the task scheduler deletes the entries whose time exceeds the 24 hours limit.

4.4 Finalized design

We'll also use **CDN (content delivery network)** in our design. We can keep images and videos of celebrities in CDN which make it easier for the followers to fetch them. The load balancer first routes the read request to the nearest CDN, if the requested content is not available there, then it forwards the request to the particular read application server (see the "load balancing chapter" for the details). The CDN helps our system to be available to millions of concurrent users and minimizes latency.
The final design is given below:

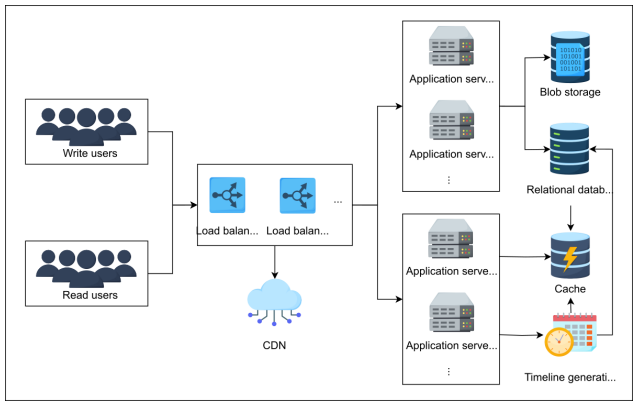


Figure 1.16: The final design of Instagram

> **Quiz: Question 1:** How can we count millions of interactions (like or view) on a celebrity post? **Answer:** We can use [sharded counters](https://www.educative.io/courses/grokking-the-system-design-interview/system-design-the-sharded-counters)
>
> to count the number of multiple interactions for a particular user. Each counter has a number of shards distributed across various edge servers to reduce the load on the application server and latency. Users nearest to the edge server get the updated count frequently on a specific post compared to those in distant regions.

4.5 Requirements compliance

Let's evaluate the Instagram design with respect to its non-functional requirements:

- **Scalability**: We can add more servers to the application service layers to make the scalability better and handle numerous requests from the clients. We can also increase the number of databases to store the growing users' data.
- **Latency**: The use of cache and CDNs has reduced the content fetching time.
- **Availability**: We have made the system available to the users by using the storage and databases that are replicated across the globe.
- **Durability:** We have persistent storage that maintains the backup of the data, so any uploaded content (photos and videos) never gets lost.

- **Consistency**: We have used storage like blob stores and databases to keep our data consistent globally.
- **Reliability**: Our databases handle replication and redundancy, so our system stays reliable and data is not lost. The load balancing layer routes requests around failed servers.

4.6 Conclusion

This design problem highlights that we can provide major services by connecting our building blocks appropriately. The scalable and fault-tolerant building blocks enable us to concentrate on use-case-specific issues (such as the efficient formation of timelines).

5 Quiz on Instagram's Design

Quiz

Question 1: If we opt for a NoSQL database to store Instagram data, which of the non-functional requirements would be **harder** to fulfill?

- Availability
- **[CORRECT]** Consistency
  Explanation: The No-SQL database is harder to keep consistent than traditional relational databases.
- Scalability
- Latency

Question 2: Which of the following options is correct to generate photos and videos for the user's newsfeed?

- **[CORRECT]** Photos or videos posted between the *last* and the current news feed generation request
- Photos or videos posted between the *first* and the current news feed generation request
- Photos or videos of the last 24 hours
- None of the above

Question 3: Which approach does our design use to generate the user's timeline?

- Pull-based
- Push-based
- **[CORRECT]** Hybrid
- None of the above